

Lab 03 — Linear Models, Built in JAX

Fundamentos de Aprendizagem Automática

Mário Antunes

2026

Overview

Class 03 · Topic 3 (first half). Lecture: `slides/03_linear_models.pdf`. Demo notebooks in `notebooks/03-linear-models/`: `01_linear_regression.ipynb` and `02_perceptron_logistic.ipynb`.

Notebook for this lab: `notebooks/03-linear-models/lab03_linear_models.ipynb`, with helpers in `lab03_utils.py` (same folder, do not edit). Cells marked **TODO** contain `raise NotImplementedError`: replace it with your code. Every other cell is ready, and it draws the plots and prints the comparisons once your functions work.

Solution: `solutions/lab03_linear_models_solution.ipynb` is the fully solved notebook. Before each function it explains how the solution was reached, and after each experiment it answers this guide's questions with the numbers the run produces. Try every task yourself first, and use the solution to check your reasoning or to get unstuck.

The rule of this lab. Every model is *a loss written with `jax.numpy`*. The gradient always comes from `jax.grad` or `jax.value_and_grad` (inside the provided `train`, an Adam optimizer with a decaying step): you never derive or type a gradient formula. Every model is then compared with its scikit-learn twin, on the **same split and the same scaling**, and the notebook prints the largest difference. A difference is not an error to hide: explain it (a regularization scaling, a solver tolerance, a converged or unconverged optimizer).

By the end of the lab you should be able to:

1. Split data correctly (stratified, k-fold), scale with training statistics only, and measure a model with the right metrics.

2. Train linear, ridge, lasso and logistic regression by differentiating a loss with JAX, and explain where each differs from scikit-learn.
3. Expand features polynomially, recognize overfitting, and control it with L_2 and L_1 penalties and a validation curve.
4. Show that the perceptron rule is a subgradient step, and why XOR defeats it.
5. Compare models with one honest protocol: folds, preprocessing inside the fold, a spread, the right metric.

Part	Task	sklearn twin	Time
A	Data, split methods, scaling, metrics from scratch	metrics, train_test_split	20 min
B	Linear regression: MSE loss, <code>jax.grad</code> , normal equation	LinearRegression	15 min
C	Polynomial features, ridge, lasso and a proximal step	PolynomialFeatures, Ridge, Lasso	30 min
D	The perceptron: its update is a <code>jax.grad</code>	Perceptron	15 min
E	Logistic regression on spam, thresholds and metrics	LogisticRegression	20 min
F	Challenge: one protocol, three models	cross_val_score	home

Setup

```
cd faa
source venv/bin/activate # see README: make venv, or pip install .
jupyter lab notebooks/03-linear-models/lab03_linear_models.ipynb
```

The Two Datasets

	Housing (regression)	Spam (classification)
Source	California housing (data/)	Spambase (data/)
Size	20640 districts, 8 features	4601 e-mails, 57 features
Target	median house value, units of 100 000 USD	$y = 1$ spam (39.4%), 0 ham
Loader	<code>load_housing()</code>	<code>load_spam()</code>
Preparation	AveRooms, AveBedrms, Population, AveOccup already replaced by their logarithm (done by the loader)	raw frequencies: you apply $\log(1 + x)$ and the scaling (Part A)

- The housing target is capped at 5.0 (500 000 USD): a limit no linear model can see.
- Most spam frequencies are exactly 0 and a few are huge. The data contain 571 exact duplicate e-mails (a leak between train and test, worth remembering when you read the accuracy).

In the notebook, `train(loss, params, *data, lr, steps)` minimizes `loss(params, *data)` and returns the final parameters and the loss history. Parameters are a dictionary `{"w": weights, "b": bias}` (a *pytree*): `jax.grad` returns a gradient with the same structure.

Part A — Data, Splits and Metrics

A1. A stratified split

Implement `stratified_split(y, test_frac, rng)`, returning `(train_idx, test_idx)` such that every class keeps the same proportion in both parts. Split each class separately, then shuffle.

1. The driver compares, on 100 e-mails, the mean gap between the spam fraction of the two parts for a random and for a stratified split. Which is smaller, and why? For which datasets does the difference matter most?

A2. Scaling on the training set only

Implement `fit_standardizer(X)` (per-column mean and standard deviation; a constant column gets standard deviation 1) and `standardize(X, mean, std)`.

1. Why must the mean and the deviation come from the **training** rows only, and be applied unchanged to the test rows? What could go wrong otherwise?

A3. Regression metrics

Implement `regression_metrics(y, y_hat)` returning `mse`, `rmse`, `mae`, `smape` (in %) and `r2`:

$$\text{sMAPE} = \frac{100}{n} \sum_i \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|} \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

The driver checks MSE, MAE and R^2 against `sklearn.metrics` and sMAPE against a hand computation (scikit-learn has no sMAPE), and prints the metrics of the “predict the training mean” baseline.

1. What are the R^2 and the RMSE of the mean baseline on the housing test set? What does $R^2 = 0$ mean?
2. When is sMAPE misleading?

A4. Classification metrics

Implement `confusion_counts(y, y_hat)` returning (tn, fp, fn, tp), and `classification_metrics(y, y_hat)` returning accuracy, precision, recall, f1 and mcc. Return 0 when a denominator is 0. The driver checks every value against scikit-learn.

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

1. The driver evaluates the useless classifier “every e-mail is spam”. Which metrics make it look acceptable, and which one tells the truth? Why?

Part B — Linear Regression

$\hat{y} = w^\top x + b$, trained by minimizing the mean squared error.

B1. Model, loss and normal equation

Implement `init_params(d)` (zeros), `predict_linear(params, X)`, `mse_loss(params, X, y)` (**no gradient formulas**) and `normal_equation(X, y)`: append a column of ones to X and solve $(A^\top A) \theta = A^\top y$ with `np.linalg.solve`, returning (w, b) .

The driver trains with `train`, fits `LinearRegression`, and prints the maximum difference of the weights, of the bias, and of the normal equation, plus the test metrics of the baseline, of your model and of `scikit-learn`.

1. Do the three solutions agree? By how much?
2. The driver trains the same model on the **unstandardized** features with the same optimizer and step count and plots both loss curves. Why is the raw version slower? (Class 02: conditioning.) Would the normal equation care?

Part C — Polynomial Features and Regularization

C1. Polynomial feature expansion

Implement `polynomial_features(X, degree)`: all monomials of the columns of degrees 1, ..., degree (no constant column), ordered by degree and, inside a degree, by `itertools.combinations_with_replacement`. The driver checks it against `PolynomialFeatures(degree, include_bias=False)` and counts the columns for degrees 1 to 4.

Then, with only **300 training examples**, it fits `LinearRegression` on the standardized expanded features for degrees 1 to 4 and plots the training and test errors.

1. How many features do degrees 1 to 4 give for the 8 housing inputs? Compare with the 300 examples.
2. Describe the two curves. Which degree would you choose, and on which data must you decide?

C2. Ridge and lasso losses

Implement `ridge_loss(params, X, y, lam) = MSE +  $\lambda\|w\|_2^2$` and `lasso_loss(params, X, y, lam) = MSE +  $\lambda\|w\|_1$` . The bias is **not** penalized. `lam` is passed to `train` as data, so JAX compiles the loss once for all values of λ .

Conventions. `scikit-learn`'s `Ridge(alpha)` minimizes $\|y - Xw\|^2 + \alpha\|w\|^2$ (a sum), so $\alpha = \lambda N$. Its `Lasso(alpha)` minimizes $\frac{1}{2N}\|y - Xw\|^2 + \alpha\|w\|_1$, so $\alpha = \lambda/2$.

The driver chooses λ by 5-fold cross-validation on the 300 examples (degree 2, 44 features; the standardizer is fitted **inside** each fold), compares your ridge with `Ridge`, shows the ridge path, repeats the experiment for degree 3 (164 features), and compares your lasso, trained by `jax.grad`, with `Lasso`.

1. Which λ does the validation curve choose, and what does its shape say about too small and too large values?
2. Compare the test error of the plain and the ridge model, for degree 2 and for degree 3.
3. Look at the lasso comparison: the objective and the predictions agree with `scikit-learn`, but not the weights. And how many weights are **exactly** zero in your lasso and in `scikit-learn`'s? Why? (Hint: the derivative of $|w|$ at 0, and the conditioning of polynomial features.)

C3. A proximal step gives exact zeros

Implement `soft_threshold(w, t) = sign(w) max(|w| - t, 0)` and `proximal_lasso(X, y, lam, lr, steps)`. Each iteration takes a gradient step on the **smooth part** (`jax.grad(mse_loss)`), then soft-thresholds the weights by `lr * lam` (the bias is not thresholded). This is ISTA. The driver uses the step $1/L$, where $L = 2\lambda_{\max}(A^\top A/N)$ is the largest curvature of the loss.

1. How many exact zeros does it give, and how many features does the lasso keep? What is its test error, against the plain fit on all 44?
2. Why does soft-thresholding produce exact zeros where the subgradient step does not?

Part D — The Perceptron

$\hat{y} = \text{sign}(w^\top x + b)$ with labels $y \in \{-1, +1\}$, trained on the **perceptron criterion** $\ell = \max(0, -m)$ with the margin $m = y(w^\top x + b)$.

D1. The loss and the online rule

Implement `perceptron_loss(params, X, y)` (mean over the rows; use `jnp.where(margin <= 0, -margin, 0.0)`, so a point on the boundary counts as a mistake) and `train_perceptron(X, y, epochs)`. It visits the examples **in order**, computes `jax.grad(perceptron_loss)` of **one example**, and updates $\theta \leftarrow \theta - g$ (learning rate 1). It returns the parameters and the number of mistakes per epoch (a mistake is a non-zero gradient).

The driver runs it on separable blobs, on overlapping blobs and on XOR, then on XOR with the degree-2 features, then on the spam data against scikit-learn's `Perceptron(penalty=None, eta0=1, shuffle=False, tol=None, max_iter=1)`.

1. Write the gradient of $\max(0, -m)$ for one example. Why is a `jax.grad` step exactly the classic rule $w \leftarrow w + yx$?
2. Describe the mistakes per epoch in the three data sets. When does the perceptron converge?
3. Why does XOR fail, and why do the degree-2 features fix it?
4. Do you get scikit-learn's weights exactly? Why does it matter that a margin of 0 is a mistake?

Part E — Logistic Regression

$p(y = 1 \mid x) = \sigma(w^\top x + b)$, trained on the **binary cross-entropy**.

E1. Loss, probabilities and regularization

Implement `sigmoid(z)`, `bce_loss(params, X, y)` for labels $y \in \{0, 1\}$ and `bce_l2_loss(params, X, y, lam) = BCE + \frac{\lambda}{2} \|w\|^2`. Write the loss with the *logits* z and `jnp.logaddexp`:

$$\ell = \log(1 + e^z) - yz$$

scikit-learn's `LogisticRegression(C)` minimizes $C \sum_i \ell_i + \frac{1}{2} \|w\|^2$, so λ corresponds to $C = 1/(\lambda N)$.

The driver trains on the standardized log-frequencies of the spam data ($\lambda = 10^{-3}$), compares weights, bias and test probabilities with `LogisticRegression`, and prints the test metrics of both.

1. How close are the two models? Do they predict the same?
2. Compare with the perceptron of Part D on the same metrics.

E2. Thresholds and the cost of a mistake

The driver plots precision, recall, F_1 and MCC against the decision threshold, then picks the **smallest** threshold whose precision on the *training* set is at least 0.99, and shows the confusion matrix and the metrics on the test set at 0.5 and at that threshold.

1. What do you gain and lose by raising the threshold? Is the precision reached on the test set the one you asked for? Why not?
2. Which data should choose a threshold?
3. Finally, the driver trains logistic regression on XOR, with and without the degree-2 features. What do the two accuracies say?

Part F — Challenge: One Protocol, Three Models

Compare **logistic regression on the log features**, **logistic regression on the raw frequencies** and **the perceptron** on spam with the same protocol:

1. stratified 5-fold cross-validation on the training set;
2. every preprocessing step (logarithm, standardizer) fitted **inside** each fold;
3. the mean and the standard deviation of the **MCC** of each model;
4. one final report of the best model on the test set, with the decision threshold chosen on a validation fold, not on the test set.

Report a table (model, mean MCC, standard deviation). How much is the logarithm worth? Is the ranking larger than the fold-to-fold spread? Why can the folds' scores not be treated as independent (Class 01: corrected resampled *t*-test)?

Going further. Use the Ames housing data (OpenML house_prices) instead of California housing, with categorical features and missing values, or add elastic net to Part C, or make the perceptron a *voted* or *averaged* perceptron and compare it with logistic regression. Naive Bayes, the probabilistic counterpart of these models, is the subject of Class 04.

Checklist

- ☐ Your split keeps the class ratio, and no statistic of the test set reaches the model.
- ☐ Your metrics equal scikit-learn's, and you can explain why MCC and F_1 disagree on “everything is spam”.
- ☐ The gradient descent, the normal equation and `LinearRegression` agree to 10^{-6} or better.
- ☐ You can explain the degree curve (train falls, test explodes) and how ridge and lasso change it.
- ☐ You know why a `jax.grad` of $\max(0, -m)$ is the perceptron rule, and why XOR fails.
- ☐ Your logistic regression matches `LogisticRegression` after the $C = 1/(\lambda N)$ mapping.
- ☐ Your protocol keeps every preprocessing step inside the folds, and you report a mean and a spread.